

INT531

Site Reliability Engineering

SLI / SLO / SLA and error budgets

Week 2 · Putting a number on “good enough”

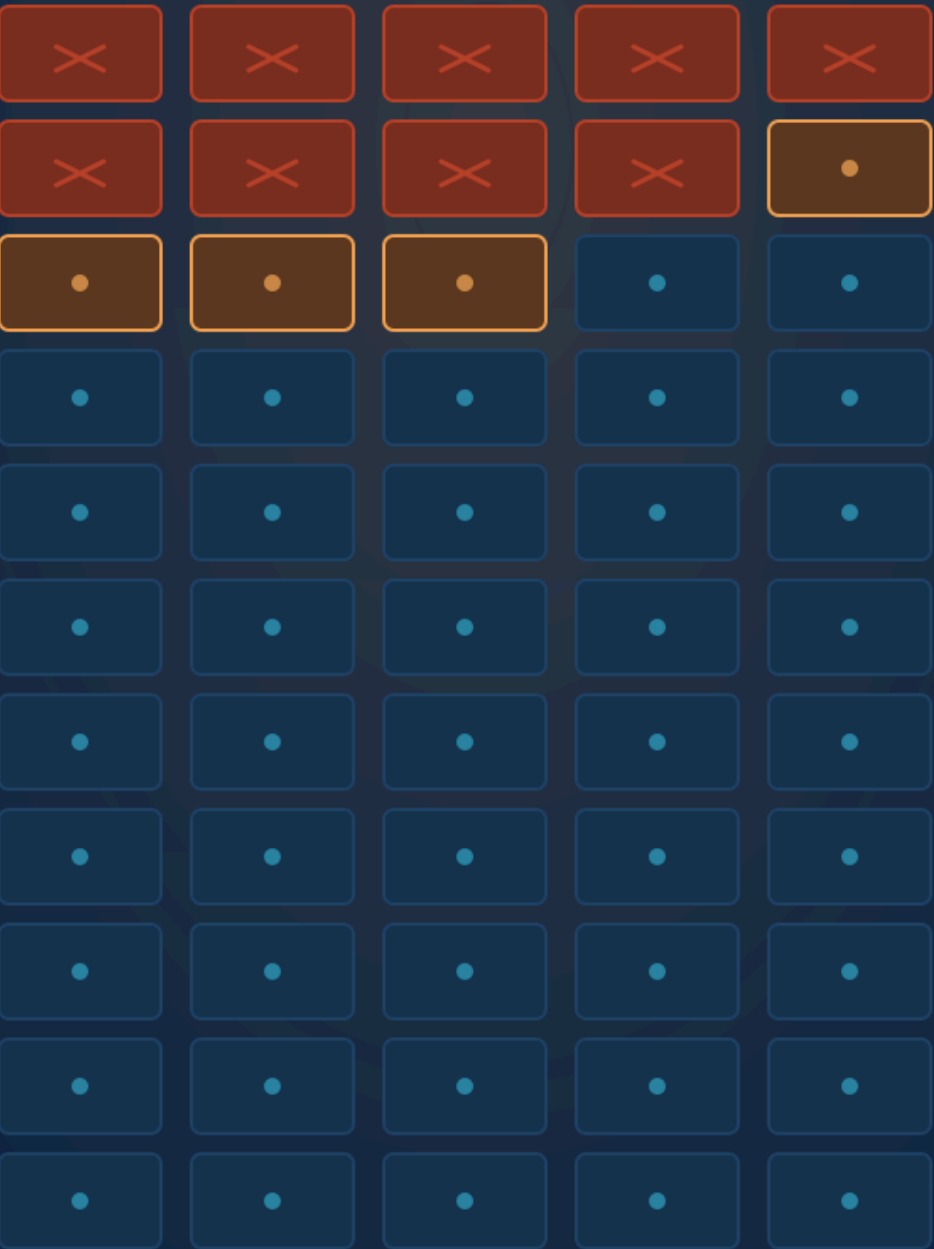
School of Information Technology, King Mongkut's University of Technology Thonburi

Lecture 2 h + Field lab 1.5 h | Content updated 2026

99.9% / 30 days

43.2 min

error budget



What we cover today

Lecture 1.5 h + field session briefing 1.5 h

01

From an open question to a number

Why “the system feels stable” cannot support any decision

02

SLI / SLO / SLA

Three terms people swap around, and what breaks when they do

03

Choosing a good SLI

An SLI menu by service type, and the trap of measuring the wrong side

04

Error budgets

The budget, the burn rate, and the policy for when it runs out

05

Lab 2 — two field sessions

Walk the containment room, then the decommission and move day

06

Room access and asset handback

The procedure under ISO/IEC 27001 and related standards

Session learning outcomes

By the end of this session you will be able to...

SLO 2.1 Tell SLI, SLO and SLA apart and give an example of each

CLO2

SLO 2.2 Choose an SLI that suits the service type, and explain why it must be measured from the user's side

CLO2

SLO 2.3 Compute an error budget from an SLO and convert it into allowable downtime

CLO2

SLO 2.4 Write an error budget policy that says what happens as the budget runs down

CLO2, CLO7

SLO 2.5 Explain why averages mislead and why percentiles are used instead

CLO2

SLO 2.6 Follow the controlled-area access and asset handback procedures correctly

CLO3, CLO7



01

From an open question to a usable number

We ended last week on one question. Today we answer it

The question left hanging last week

“If you had one number to say whether the registration system is good enough, what would it be?”

Answers that sound fine but decide nothing

“99.9% uptime”

Uptime of what, measured where? If the server returns 200 but the page never renders, does that still count as up?

“CPU averages 40%”

That is a number about the machine, not the user. Idle CPU while nobody can log in happens all the time.

“Nobody has complained”

Does that mean there is no problem, or that users have given up? We cannot tell those apart.

An answer you can act on

“Over the last 30 days, 99.2% of registration submissions succeeded within 2 seconds — below the 99.5% target we set.”

That one sentence says what was counted, from whose side, over what window, and against what target.

SLI, SLO, SLA — never use these interchangeably

Each has a different owner, a different audience, and a different consequence



The most common mistake is setting the SLO exactly equal to the SLA — at which point missing your internal target once is immediately a breach of contract.

The anatomy of an SLI, and the equation behind it

Every usable SLI is a ratio of events — if you cannot write it as a fraction, it is not defined yet

SLI

=

good events

valid events

×

100 %

Requests that met the success criterion

All requests that should have been served

Worked example — registration submissions, 30-day window

Total submissions received	205,140
Health checks and monitoring probes (excluded)	– 5,140
Valid events (denominator)	<u>200,000</u>
Returned 2xx within 2 s (numerator)	<u>198,412</u>
SLI	99.206 %

Excluding events from the denominator is a decision, not a detail. Write down every exclusion — that is where SLIs quietly become dishonest.

From a sentence to something measurable (specification → implementation)

Every SLO needs both layers, or two people will measure two different numbers

Layer	What it contains	Registration system example
Specification	The meaning, in plain language, with no tooling detail	The proportion of registration submissions the system serves successfully within an acceptable time
Implementation	The formula actually measured: where it is collected, what is counted, and the thresholds	At the load balancer: requests with status < 500 and duration ≤ 2s, divided by all requests to path /register
Exclusions	Events that do not enter the denominator, each with a stated reason	Health checks, monitoring probes, and requests the user cancelled themselves (client abort)

Why two layers

The specification is what you discuss with users and management; the implementation is what an engineer builds. With only one, the argument about what “success” means never ends

Where you measure matters as much as what

Measure in the application and you miss load balancer failures; measure at the load balancer and you miss DNS failures. The closer to the user, the closer to the truth

Write the exclusions down

Every exclusion makes the number look better, so each one needs a stated reason and periodic review — not a quiet addition when the number looks bad

Choosing an SLI that suits the service

Do not measure what is easy to measure; measure what the user feels

Service type	What the user cares about	SLIs to use	Example on campus
Request / response	It works when I click, and fast enough	Availability, latency, correctness	Registration, LMS, faculty APIs
Batch pipeline	The data arrives complete and on time	Freshness, coverage, correctness	Grade rollups, student data sync
Storage	What I wrote can always be read back	Durability, availability, latency	Shared file service, backup system

Three traps that make an SLI look good and mean nothing

Measuring only at the server

The server reports 200 for everything, but if the CDN or load balancer is in the way the user never got it — the SLI looks healthy during an outage

Counting only the requests that arrive

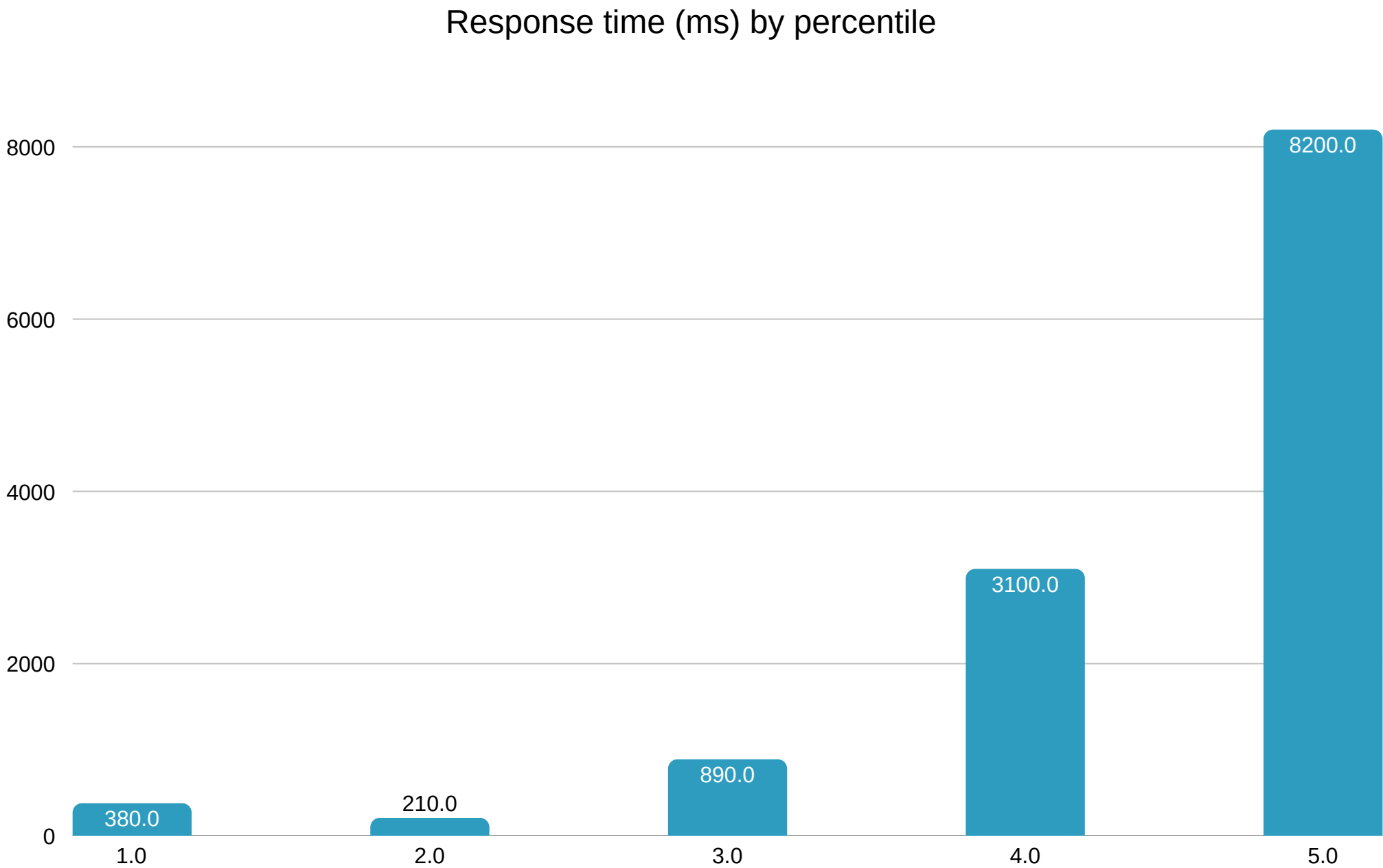
In a severe outage many requests never reach your counter at all, so the success rate rises exactly when things are worst

Using an average instead of a percentile

An average always hides the users having the worst time. See the next slide

Why the average lies

Illustrative data from the registration system on opening day



An average of 380 ms passes comfortably

Report that number upward and everyone relaxes

But 1 user in 100 waits over 3 seconds

On a day with 200,000 requests that is 2,000 very slow ones

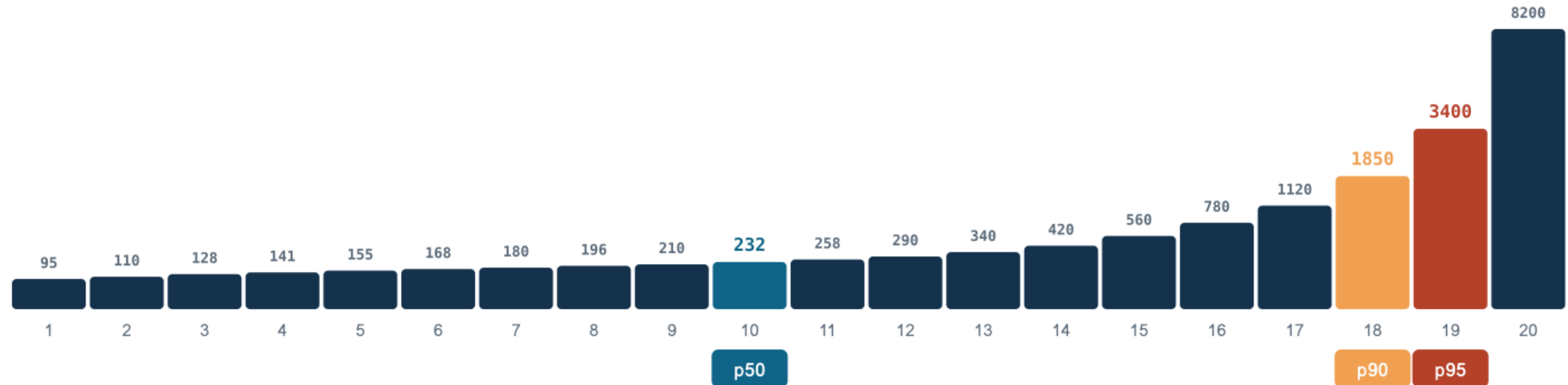
And 1 in 1,000 waits over 8 seconds

These are the people who call to complain, and who retry until the system gets worse

Rule of thumb: always state an SLO as a percentile, and pick the percentile to match how many real users sit behind it

Computing a percentile, step by step

Three steps, and the one rule people break most often



You need enough samples

To talk about p99 you need at least 100 samples in the window, and p99.9 needs at least 1,000. Below that the number means nothing

Pick the percentile to match your user count

With 20,000 users, p99 still means 200 people having a bad time. Large user bases need p99.9

Histograms, not raw values

Real systems bucket the data and estimate percentiles from the buckets, so the value is an approximation, not exact

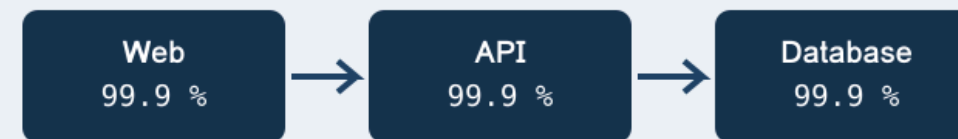
You cannot average percentiles. The mean of two servers' p99 is not the p99 of the pair — recompute from the raw samples.

Composite availability

Your SLO is always capped by what you depend on

Combining components changes the number

In series — every hop must work



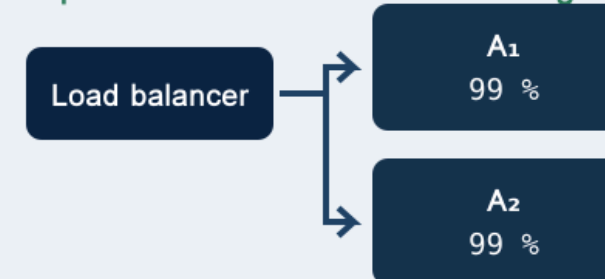
$$A_{\text{total}} = A_1 \times A_2 \times A_3$$

$$0.999 \times 0.999 \times 0.999 = 0.997$$

99.7 % → 2 h 10 min / 30 days

Three 99.9% services chained give less than any one of them

In parallel — one survivor is enough



$$A_{\text{total}} = 1 - (1 - A_1) \times (1 - A_2)$$

$$1 - (0.01 \times 0.01) = 0.9999$$

99.99 % → 4.3 min / 30 days

Two 99% servers behind a load balancer give more than either one

Rule one

Never set an SLO above the product of your dependencies. If the database gives 99.9%, you cannot promise 99.95% unless you are designed to keep working without it

Rule two

Adding a component in series always reduces reliability. This is why the simpler system is usually the more reliable one

Rule three

Redundancy only helps if the two do not fail together. If both sit on the same power feed, the parallel formula does not apply



02

Error budgets

The one tool that lets development and operations argue with numbers instead of feelings

The arithmetic of an error budget

A budget is time you are allowed to spend, not time you must avoid

SLO	Per 30 days	Per 7 days	Per day	Suits
99%	7 h 12 min	1 h 41 min	14 min 24 s	Internal, non-critical systems
99.5%	3 h 36 min	50 min 24 s	7 min 12 s	General services
99.9%	43 min 12 s	10 min 5 s	1 min 26 s	Systems with external users
99.95%	21 min 36 s	5 min 2 s	43 s	Registration at peak

The budget covers everything: unplanned outages, bad deploys, and planned maintenance alike

Ending the month with most of it unspent means you were too cautious and probably shipped too slowly

Running out mid-month means stop adding risk now, not at the end of the period

Computing an error budget, step by step

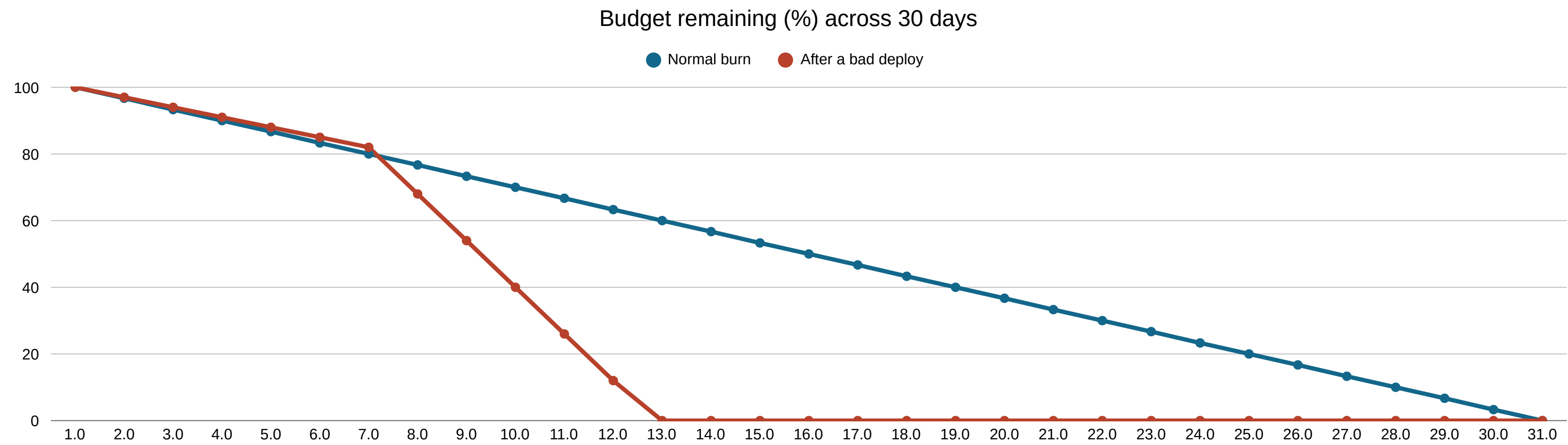
Worked example: registration system, SLO 99.9%, 30-day window

1	Fix the window	$30 \text{ days} = 30 \times 24 \times 60$	43,200 min
2	Budget as time	$\text{budget} = (1 - \text{SLO}) \times \text{window} = 0.001 \times 43,200$	43.2 min
3	Budget as requests	$\text{budget} = (1 - \text{SLO}) \times \text{total requests} = 0.001 \times 200,000$	200 requests
4	Check what was spent	Requests that actually failed this period = 1,588	794 % of budget
5	State the position	Overspent by 7.9x → stop shipping now, per the policy	Budget exhausted

Note that 1,588 comes straight from the SLI anatomy slide (200,000 – 198,412) — the whole session runs on one set of numbers

Burn rate

Watch the speed the budget is draining, not the balance left



Burn rate = 1

Spending in step with the clock. The budget runs out exactly on the last day of the period. This is normal

Burn rate = 6

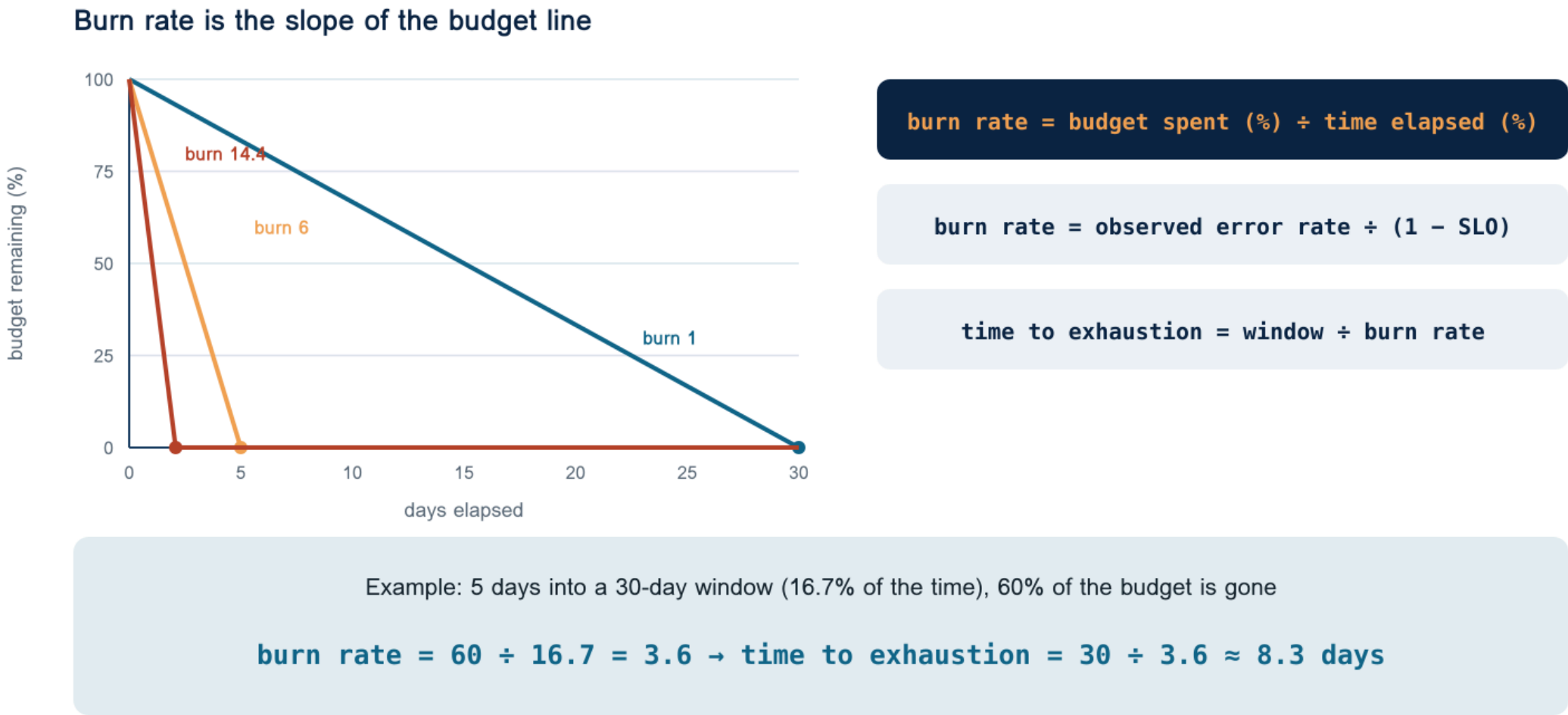
Six times faster than normal, gone in 5 days — raise a ticket to be handled in working hours

Burn rate = 14.4

Gone in 2 days — this one wakes somebody up. It is where the multi-window alerts in week 6 come from

Burn rate — the equations and the exhaustion table

Three usable equations, and the table that becomes alerting rules in week 6



Burn rate	Observed error rate (SLO 99.9%)	Budget gone in	What to do
1	0.1 %	30 days	Normal. Do nothing
2	0.2 %	15 days	Watch it; note it in the weekly report
6	0.6 %	5 days	Raise a ticket for working hours
14.4	1.44 %	2 days	Page someone now

The error budget policy — agreed before the budget runs out

Written after the fact, it just becomes an argument about who is to blame

Over 50% left

Ship as normal, and use the room to try things — this is when to run a chaos experiment

25–50% left

Review high-risk changes and tighten pre-deploy testing

Under 25% left

Pause new features. Only changes that reduce risk go out

Budget exhausted

Stop shipping. The team moves to reliability work until the next period, and a postmortem is required

This policy has to be agreed by development, operations and the product owner before it is needed — not announced by one side

One month, worked through — from numbers to decisions

Registration system, SLO 99.9% · 43.2 minutes for the whole period

Day	Event	Spent	Total	Left	Decision under the policy
3	Deploy v2.4 causes errors	5.0 min	5.0	88 %	Ship as normal
9	Database failover	12.0 min	17.0	61 %	Ship as normal
16	Planned maintenance window	8.0 min	25.0	42 %	Review high-risk changes
22	Bad deploy, rolled back	13.0 min	38.0	12 %	Pause new features
27	Network incident	6.0 min	44.0	– 2 %	Stop shipping + postmortem

The key point: the planned maintenance on day 16 came out of the same pot. Users do not care whether the downtime was intentional



03

Lab 2 — two field sessions

Walk the containment room · then strip the old racks, inventory everything, and hand back what is unused

Why this sits in the same week as error budgets

Because stripping the racks spends the largest single piece of budget this term

What does this work cost?

Every minute a service is down during the move is a direct withdrawal. The team estimates it in advance and checks it against the budget

Why out of hours?

So the downtime lands when request volume is lowest, which is when a request-based SLI takes the least damage

Why inventory everything?

An item not on the register is an item nobody owns. When it fails nobody gets paged, and nobody knows where its data went

Why hand back what is unused?

Hardware drawing power and occupying U space while serving nobody is a cost paid every month for nothing

Field session schedule

Put the real dates on the access request and confirm with computer centre staff at least five working days ahead

Two field sessions — both inside the course timetable

Session A · Site walkthrough

Week 2 lab slot · 13:00–16:00 · new containment room

13:00	13:30	14:30	15:30
Access briefing and sign-in	Walk the containment room	Measure U space, power and cooling	Agree the target layout

Session B · Decommission and move

Week 3 Saturday · 09:00–16:00 · old server room (out of hours)

09:00	09:30	10:30	12:00	13:00	14:30	15:30
Sign-in, escort assigned, safety brief	Photograph and label everything	Power down in dependency order	Break	Remove equipment, record every serial	Separate keep / return / dispose	Hand over to the asset officer, sign out

No entry without an approved access request

Actual dates: Session A · Session B · Access request due

Getting into a controlled area

Every step leaves evidence · ISO/IEC 27001:2022 controls A.7.1–A.7.4

1	File the request	Name, student ID, purpose, date and time, and the tools being brought in — at least five working days ahead	A.7.2
2	Get it approved	The instructor endorses it and the area custodian approves. Access is limited to the stated date and time	A.7.2 / A.5.15
3	Sign in and meet your escort	Show ID, sign the log, and stay with the escort throughout. Nobody is alone in the room	A.7.2 / A.7.4
4	Work inside the stated scope	Only what the request covers. Photography is authorised case by case	A.7.4 / A.5.37
5	Sign out and hand over	Count the tools in and out, hand the asset records to staff, and sign out	A.7.4 / A.5.11

Bringing in anyone not named on the approved request is a control violation and costs the whole group marks

Session A — walking the containment room

Three hours · what to measure and what to come back with

Measure these on site

- Actual free U in each rack, and where blanking panels already are
- Each PDU's rating, outlet count and connector type
- Distance from rack to patch panel, for cable length
- Airflow direction in and out, and where the containment panels sit
- Door and aisle height and width, for the move route

Come back with these

- A draft target layout the whole group has agreed
- A list of cables to procure, with lengths and quantities
- Expected electrical load per PDU and per phase
- Site constraints you found that were not in the original plan
- Authorised overview photographs

Session B — decommission and move day

An order you do not rearrange • every step has a named signer

- 1 Before touching anything** Photograph the original state from every side, label both ends of every cable, and confirm a current backup exists for every system about to be powered down
- 2 Declare the start** Notify users through the agreed channel, record the start time, and begin counting error budget spend
- 3 Power down in dependency order** Work down from the top of the dependency map. Anything others depend on goes last
- 4 Remove and record** One item at a time. Serial, original U position and condition go onto the form immediately — never written up afterwards
- 5 Sort each item onto its path** Group into keep / return / dispose and colour-tag each item the moment it is sorted
- 6 Hand over and close out** Hand the list to the asset officer with both signatures, record the end time, and total the budget actually spent

Building the asset register during the strip-out

Recorded at the point of removal, not reconstructed later

Identity

Make, model, serial number, faculty asset tag, and the MAC address of the first port

Where it came from and goes

Original U position, original rack, and its destination path (keep / return / dispose)

Operational status

Services running before shutdown, the owner, and the in-service date if known

Media inside it

Number and type of disks, capacity, and sanitisation status

Physical condition

Damage, missing parts, and at least one photograph

Who recorded it

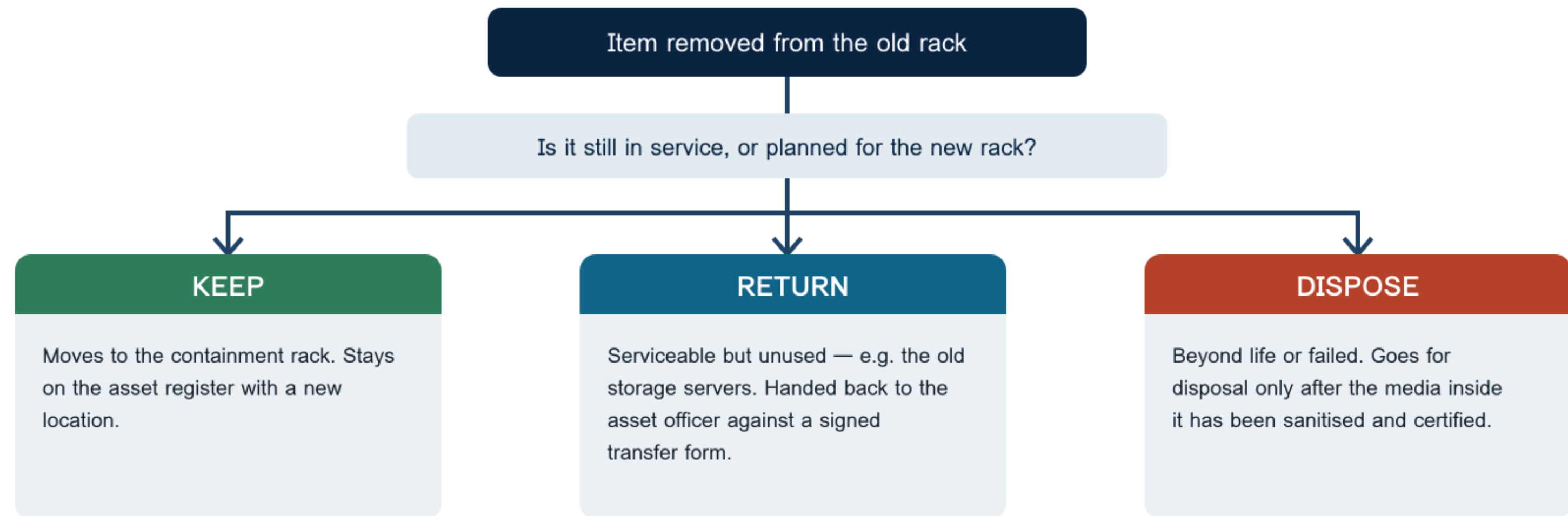
Who removed it, who checked it, and the time recorded

One item, one line. Never group them — ten identical units still have ten serials and are ten separate assets

Where each asset goes after it leaves the rack

Every item needs a traceable path and a signature

Every item leaves the old room down exactly one of three paths



Any item in the RETURN or DISPOSE path that contains a disk, SSD or memory card must be sanitised first — and the certificate filed

[A.5.9 asset register](#)

[A.5.11 return of assets](#)

[A.7.10 storage media](#)

[A.7.14 secure disposal or re-use](#)

Handing back unused equipment — the old storage servers

A real case they will hit on move day

- 1 Confirm nobody is actually using it** Check historical access logs, ask the system owners, and confirm no service still points at it
- 2 Preserve any data still under retention** If data must still be kept, move it off and verify it reads back before going any further
- 3 Sanitise the disks** Use a method appropriate to the media type, and record the method, the operator and the timestamp on a sanitisation certificate
- 4 Update the asset register** Change the status from in-service to pending return, with the reason and the approver
- 5 Hand over with both signatures** Transfer to the asset officer on a signed form stating quantity and condition, with the sanitisation certificate attached

Never hand back or scrap a device with disks still inside it and no sanitisation certificate. This is the most common data-leak path in equipment retirement

The controls this field work maps to

In addition to the ones already used in week 1

A.5.11	Return of assets	Unused equipment goes back to the asset officer on a form signed by both parties
A.5.15	Access control	Entry rights are limited to the named people, date and time on the request
A.7.1 / A.7.2	Secure areas and physical entry	An escort throughout; nobody is ever alone in the room
A.7.4	Physical security monitoring	Entry and exit logged; photography authorised case by case
A.7.10	Storage media	Every disk removed is controlled from removal until sanitisation is complete
A.7.14	Secure disposal or re-use of equipment	Sanitise before return or disposal, with a certificate
A.8.32	Change management	An approved change request stating the window, the impact and the rollback plan

Related standards: ISO/IEC 27002:2022 · ISO/IEC 22237 (data centre facilities) · NIST SP 800-88 Rev.1 (media sanitisation) · university procurement rules

Safety and roles on move day

Heavy lifting inside a controlled area · everyone knows their role before entering

Safety rules — no exceptions

- Some units exceed 20 kg — always two people, and use the trolley provided
- No cable comes out until the change owner gives the signal, even if the box looks dead
- Closed shoes, no long metal jewellery, hair tied back
- Watch for sharp rack edges and cable trays. Gloves are at the sign-in point
- Unusual heat, a burning smell or sparks: step back and report immediately
- The safety officer can stop the work instantly, without asking anyone

Team roles on move day

Change owner

Decides to proceed or abort, and signals the start of each step

Inventory lead

Holds the forms and checks every item is recorded before it leaves the room

Safety officer

Owens lifting and cable handling, and can call a stop

Scribe

Records start and end times per step, and anything that went wrong

What you submit from this week

One individual piece and three group pieces

Assignment 1 (individual)	An SLO document for your chosen service: the SLI, how it is measured, the target, the window and the reasoning	5%	Week 4
Site walkthrough record (group)	All five sets of on-site measurements, plus the draft target layout	3%	3 days after session A
Complete asset register (group)	One item per line, with destination path and supporting photographs	5%	5 days after session B
Asset handback pack (group)	The transfer form signed by both parties, with sanitisation certificates for every item containing media	2%	5 days after session B

Assignments and next week

Week 3: systems and distributed systems fundamentals for reliability

Before the next session

- Read Google SRE Book chapter 6 (Monitoring Distributed Systems)
- Complete pre-class quiz 2 on the LMS
- Revise the Linux tools: top, vmstat, iostat, ss

Field work to prepare

- File the access requests for both sessions this week
- Assign all four roles within the group and name your coordinator
- Kit: tape measure, torch, cable labels, camera, and the forms printed

Carrying on from Lab 1

- Update the dependency map with what you find on the walkthrough
- Estimate the error budget the move day will spend
- Derive the power-down order from the dependency map

The question that opens next session: if a system gets slower but has not failed, how do we know it is about to — before the users do?

References

Split by part of the session

For the lecture

- Beyer, B. et al. (2016). Site Reliability Engineering. O'Reilly. Ch. 4 (Service Level Objectives)
- Beyer, B. et al. (2018). The Site Reliability Workbook. O'Reilly. Ch. 2 and 5
- Hidalgo, A. (2020). Implementing Service Level Objectives. O'Reilly. Ch. 3–5

For the field work

- ISO/IEC 27001:2022 — Annex A: A.5.11, A.5.15, A.7.1–A.7.4, A.7.10, A.7.14, A.8.32
- ISO/IEC 27002:2022 — implementation guidance for those controls
- ISO/IEC 22237 series — Data centre facilities and infrastructures
- NIST SP 800-88 Rev.1 — Guidelines for Media Sanitization
- Your university's procurement forms for returning and disposing of capital equipment

Today in three sentences

01

If you cannot say what good enough is, measured from whose side, over what window, then you have not set a target

02

An error budget turns “whose fault is it” into “how much risk can we still afford” — and that question has a numeric answer

03

Stripping the racks next week spends a real error budget. Every minute of downtime comes out of the same pot

Week 3 · Systems and distributed systems — we go find where the number went bad